

Christian Aiello

CMPSC 132

Professor Kambhampaty

28 April 2026

Number Guessing Game – Final Project Report

This is a terminal based Number Guessing Game written in Python. The program is to generate a random number and allow the user to guess it, with multiple attempts. The program, after each guess, tells the player if the guess was too high, too low, or correct. It's a game until the right number is guessed. Besides basic functionalities, the program offers optional improvements such as difficulty levels and the ability to replay the game. These features increase user interaction and make the game more engaging. The structure remains clear and simple.

The program is separated into a few functions to structure the logic and improve readability. The `main()` function drives the overall logic of the program, such as starting the game, running each round and asking the user if they want to play again. At the beginning of each round the user chooses a difficulty level and this gives a range of numbers. The next step is to use `random.randint()` to pick a secret number from the range the player chose. A loop is used to repeatedly ask the user to guess until the correct number is entered. The program gives feedback after each guess, telling the player if the guess was too high or too low, and keeps track of the number of guesses. When the correct guess is made the program will print a success message and the number of attempts.

The project is mostly built on primitive data types like integers and strings. The secret number, user guesses and the number of attempts are stored as integers, while strings are used for input from the user and output messages. Constants are also used for defining difficulty ranges, which improves organization and readability of code. Control structures are important to control the flow of the game such as loops and conditional statements.

The main challenge in this project was to handle the user input properly. The `input()` function always returns a string, so it was necessary to convert this to an integer, and also handle possible errors. `try/except` blocks were used to ensure an invalid input would not crash the program. The program was also a challenge to hack in a number of functions. This required more planning up front, but it made the code easier to read and debug in the end. The control flow had to be carefully designed so that the game loop would work correctly and could be repeated.

There's a lot of room to make this project better next time. For starters, limiting the number of guesses would crank up the challenge. Adding a scoring system that rewards quick thinking is also a solid idea. It'd also help if the game could save top scores to a file, so people don't lose their progress. A fresh look with a graphical interface would make it way more inviting to play. And honestly, giving players hints about how close their guess is would just make everything more fun.

Overall, this project demonstrates important programming concepts such as loops, conditionals, functions, and input validation. It successfully meets the requirements of the assignment while also including additional features that improve usability. The project provided valuable practice in structuring a program and handling errors, and it serves as a strong foundation for more advanced programming work in the future.